

WebSockets for Absolute Beginners

Lesson 3: Setting up a WebSocket Server (Node.js Example)

Setting up a WebSocket Server (Node.js Example) - Study Notes

Key Concepts

- **WebSockets:** A communication protocol that provides full-duplex communication channels over a single TCP connection. Unlike HTTP, WebSockets maintain a persistent connection, allowing for real-time data exchange.
- **Node.js:** A JavaScript runtime environment that allows you to execute JavaScript code outside of a web browser. It's well-suited for building network applications like WebSocket servers.
- **`ws`` Library:** A popular Node.js library that simplifies the creation and management of WebSocket servers and clients.
- **Bidirectional Communication:** WebSockets enable both the server and the client to send data to each other simultaneously, unlike the request-response model of HTTP.
- **Event-Driven Architecture:** Node.js and the `ws`` library utilize an event-driven architecture, where code responds to specific events (like a new connection or a message received).

Summary

This lesson introduces the fundamentals of building a WebSocket server using Node.js and the `ws`` library. WebSockets offer a significant advantage over traditional HTTP for real-time applications because they establish a persistent connection, eliminating the overhead of repeated HTTP requests. The example code demonstrates how to set up a basic server that listens for incoming connections, logs connection events, and echoes back any messages received from clients.

The core of the server involves creating a WebSocket server instance and defining event handlers for ``connection``, ``message``, and ``close`` events. The ``connection`` handler is triggered when a new client connects, the ``message`` handler processes incoming messages, and the ``close`` handler is invoked when a client disconnects. This simple structure provides a solid foundation for building more complex real-time applications, such as chat applications, live dashboards, or online games.

Important Points

- **Installation:** You must install the `ws`` library using ``npm install ws`` before running the server.
- **Server Creation:** The `ws`` library provides a simple constructor to create a WebSocket server:
``const wss = new WebSocket.Server({ port: 8080 });``
- **Event Handlers:**
 - * ``connection``: Called when a new client connects to the server. Useful for logging connection events or performing initial setup.
 - * ``message``: Called when a message is received from a client. This is where you process the incoming data.
 - * ``close``: Called when a client disconnects from the server. Useful for cleanup tasks.
- **Message Echoing:** The example echoes messages back to the client using ``ws.send(message)``.

- **Error Handling:** While not explicitly detailed in the lesson content, robust applications should include error handling within the event handlers to gracefully manage unexpected situations.
- **Port Selection:** Choose a port number that is not already in use by another application. 8080 is a common choice for development.

Examples

Example 1: Basic Server Setup

```
```javascript
const WebSocket = require('ws');
const wss = new WebSocket.Server({
 port: 8080
});
console.log("WebSocket server started on port 8080");
wss.on('connection', ws => {
 console.log('Client connected');
 ws.on('message', message => {
 console.log(`Received: ${message}`);
 ws.send(`You sent: ${message}`); // Echo back the message
 });
 ws.on('close', () => {
 console.log('Client disconnected');
 });
});
```
```

Explanation: This code creates a WebSocket server listening on port 8080. When a client connects, a message is logged to the console. Any message received from the client is logged and then echoed back to the client. When the client disconnects, a message is logged.

Example 2: Sending a Message to All Connected Clients

```
```javascript
const WebSocket = require('ws');
const wss = new WebSocket.Server({
 port: 8080
});
console.log("WebSocket server started on port 8080");
wss.on('connection', ws => {
 console.log('Client connected');
 ws.on('message', message => {
 console.log(`Received: ${message}`);
 wss.clients.forEach(client => {
 if (client !== ws && client.readyState === WebSocket.OPEN) {
 client.send(`Message from ${ws.url}: ${message}`);
 }
 });
 });
});
```
```

```

});
ws.on('close', () => {
  console.log('Client disconnected');
});
});
```

```

**\*Explanation:** This example extends the previous one. When a message is received, it iterates through all connected clients (excluding the sender) and sends the message to each client that is currently open. `ws.url` provides the URL of the connecting client.

## Quick Review

1. What is the primary advantage of using WebSockets over HTTP for real-time communication?
2. What is the purpose of the `ws` library in Node.js?
3. Describe the function of the `connection` event handler in a WebSocket server.
4. What does the `message` event handler do?
5. Why is it important to check `client.readyState === WebSocket.OPEN` before sending a message to a client?

## Further Reading

- **`ws` Library Documentation:** [<https://github.com/websockets/ws>](<https://github.com/websockets/ws>) - The official documentation for the `ws` library.
- **WebSockets Protocol Specification:** [<https://tools.ietf.org/html/rfc6455>](<https://tools.ietf.org/html/rfc6455>) - The formal specification for the WebSocket protocol.
- **Real-time Communication with WebSockets:** [<https://www.freecodecamp.org/news/real-time-communication-with-websockets/>](<https://www.freecodecamp.org/news/real-time-communication-with-websockets/>) - A comprehensive article on WebSockets and their applications.
- **Node.js Event Loop:** Understanding the Node.js event loop is crucial for building efficient and scalable WebSocket applications. Search for "Node.js event loop" for more information.

